

Transcendence

Un Tock à 4 joueurs en ligne — architecture, code et modules

Support de soutenance · ft_transcendence (sujet v21.1)

Ce qu'on va voir

1

La big picture

Le jeu, la stack, l'architecture Docker / nginx

2

Backend NestJS

Auth, profil, amis, base de données

3

Le moteur de jeu

Plateau, cartes, règles, IA — du code pur

4

Temps réel

WebSockets, diffusion de l'état, déconnexions

5

Frontend Next.js

Pages, hooks, plateau, design system

6

Grille & modules

Exigences, points, ce qu'il reste à corriger

1

La big picture

Ce que fait l'application et comment les morceaux s'emboîtent

Un Tock à 4 joueurs, jouable dans le navigateur

4 joueurs, 4 pions chacun sur un plateau de 72 cases ; 52 cartes, 5 par manche.

Les sièges vides sont pris par des IA : on peut jouer seul ou entre amis.

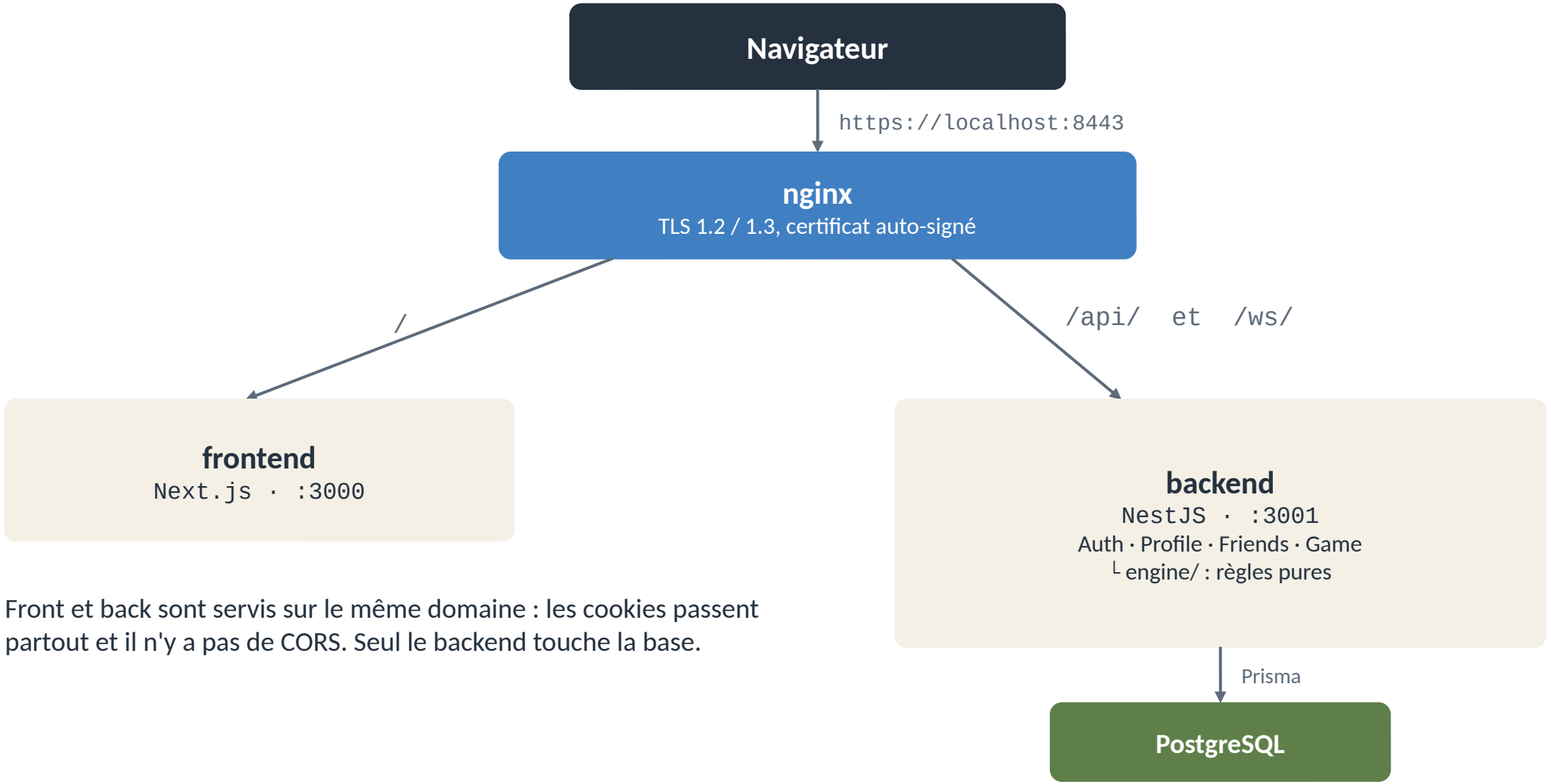
Temps réel : chaque coup est poussé à tous les joueurs par WebSocket.

Victoire : le premier qui rentre ses 4 pions dans sa maison.

Autour du jeu : comptes, profil + avatar, amis avec statut en ligne.

Couche	Techno	Pourquoi
Frontend	Next.js 16, React 19	Routing par fichier, écosystème React
Style	Tailwind 4 + shadcn/ui	Tokens de couleur, composants accessibles
Backend	NestJS 11 + socket.io	Modules + injection, gateways WebSocket intégrées
ORM / BDD	Prisma 7 + PostgreSQL 18	Schéma typé, migrations, contraintes
Proxy	nginx	TLS + un seul point d'entrée
Déploiement	Docker Compose	Tout démarre en une commande

Le navigateur ne parle qu'à nginx, qui aiguille selon l'URL



Front et back sont servis sur le même domaine : les cookies passent partout et il n'y a pas de CORS. Seul le backend touche la base.

Une seule commande : docker compose up --build

`postgres`

Image officielle, volume db_data, healthcheck pg_isready

`backend`

Attend postgres « healthy », puis migrate deploy → generate → start

`frontend`

next dev, code monté en volume (rechargement à chaud)

`nginx`

Génère le certificat avec openssl au build, écoute 443 → 8443

docker-compose.yml

```
backend:
  # A started container is not a ready database: migrate deploy
  # would fail on a fresh volume
  depends_on:
    postgres:
      condition: service_healthy
  …
  healthcheck:
    # $$ so the variables are read inside the container, not by
    # compose
    test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d
        $${POSTGRES_DB}"]
    interval: 2s
```

backend/Dockerfile

```
CMD ["sh", "-c", "npx prisma migrate deploy
  && npx prisma generate && npm run start:dev"]
```

Sans le healthcheck, le backend lancerait les migrations sur une base pas encore prête.

Quatre principes guident tout le code

1 Le serveur fait autorité

Le front envoie une intention (« cette carte, ce pion »). Le serveur la valide ou la refuse : aucune règle n'est appliquée côté client.

2 Le moteur est pur

engine/ ne contient que des fonctions (état, action) → nouvel état. Ni Nest, ni Prisma, ni mutation : facile à tester et à relire.

3 La partie vit en mémoire

Une Map dans GameService. La base ne garde que ce qui dure : comptes, profils, parties, participants, amitiés.

4 Un seul point d'entrée

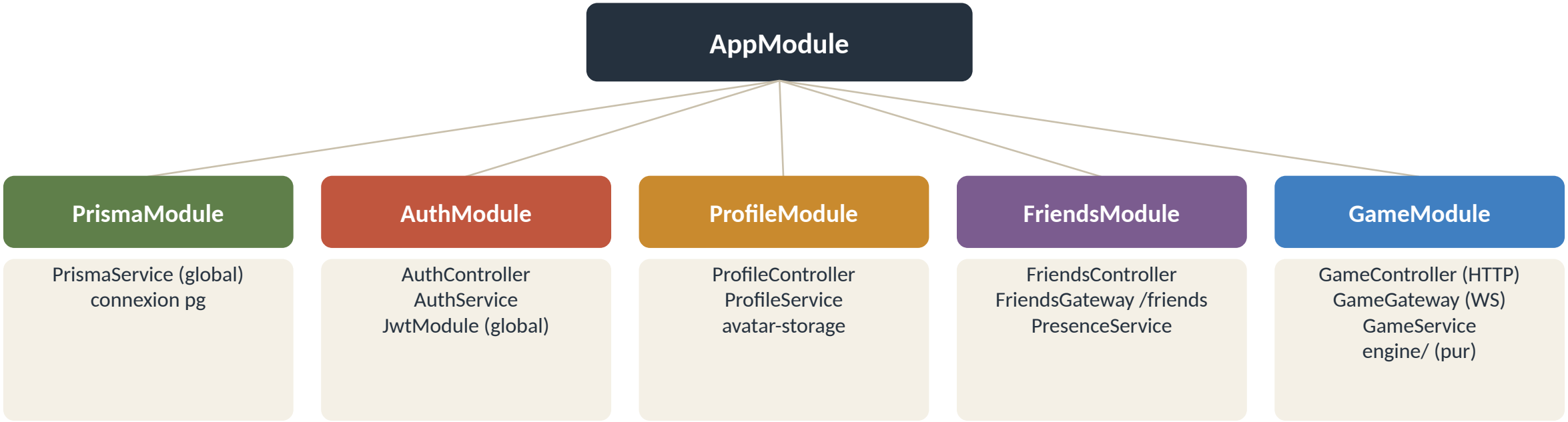
Tout passe par nginx en HTTPS. Les cookies httpOnly suivent les requêtes HTTP et le handshake WebSocket.

2

Backend NestJS

Un module par domaine, tout sous /api

NestJS : un module par domaine, branchés dans AppModule



backend/src/main.ts

```
app.setGlobalPrefix('api');

app.use(cookieParser());
```

- Préfixe global /api : nginx n'a qu'une règle pour le backend.
- JwtModule global : JwtService injectable aussi dans les gateways WebSocket.
- Les avatars sont servis en statique, avec nosniff.

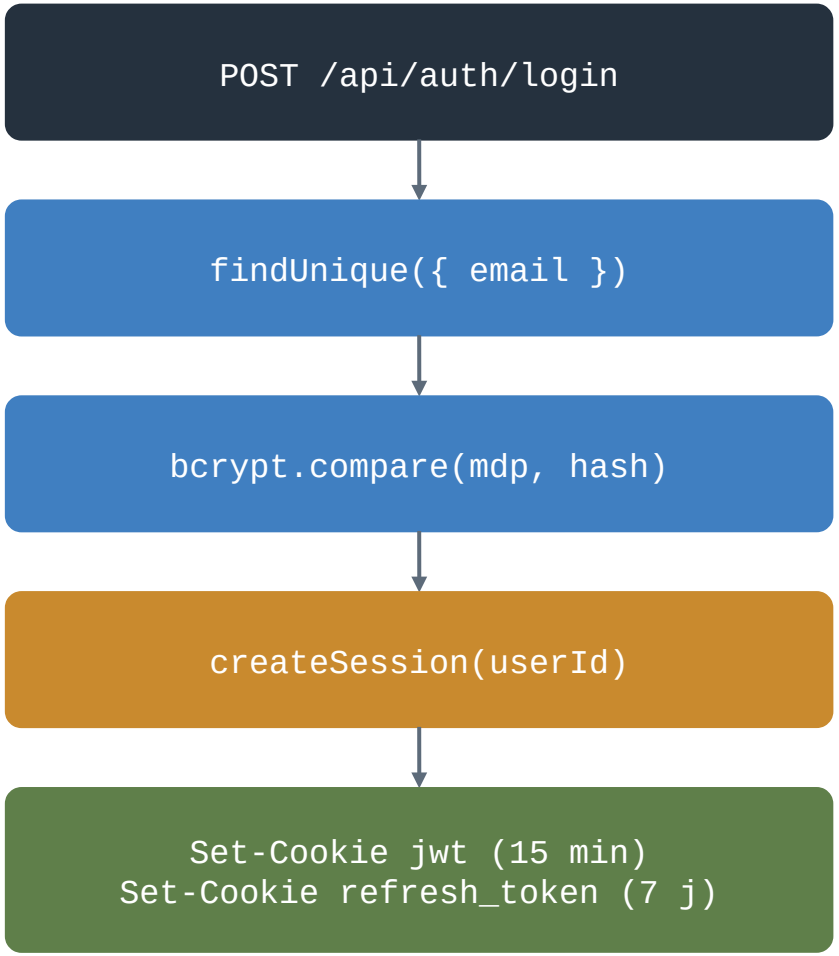
Les routes HTTP (toutes sous /api)

Méthode	Route	Auth	Rôle
POST	/auth/register	—	Crée le user et un profil vide
POST	/auth/login	—	Pose les cookies jwt + refresh_token
POST	/auth/refresh	cookie	Rotation des deux tokens
PATCH	/auth/password		Change le mot de passe
POST	/auth/logout		Révoque le refresh token
GET	/profile		Profil + stats (amis, parties)
PATCH	/profile		Pseudo, bio
POST	/profile/avatar		Upload (2 Mo max)
DELETE	/profile/avatar		Retour à l'avatar par défaut

Méthode	Route	Rôle
GET	/friends	Amis, demandes reçues / envoyées
POST	/friends/requests	Demande par pseudo
POST	/friends/requests/:id/accept	Accepter
DELETE	/friends/:id	Retirer / refuser
POST	/game	Crée une partie (siège 0)
POST	/game/join/:inviteCode	Rejoint une partie
POST	/game/start/:gameId	L'hôte lance, IA en renfort

Amis et jeu : tout est protégé par AuthGuard. Les routes de jeu ne renvoient jamais l'état de la partie : il arrive toujours par WebSocket.

Login : bcrypt pour le mot de passe, deux JWT en cookies



backend/src/auth/auth.service.ts

```
{
    const salt = await bcrypt.genSalt(10);
    const hashPassword = await bcrypt.hash(data.password, salt);
  …
async login(data: {email: string; password: string;})
{
    const user = await this.prisma.user.findUnique({
        where: {
            email: data.email,
        }
    });
    if (!user)
        throw new UnauthorizedException();

    const password_comp = await bcrypt.compare(data.password,
        user.password);
    if (!password_comp)
        throw new UnauthorizedException();

    return this.createSession(user.id, user.name);
```

Refresh token : rotation à chaque usage, révocable en base

backend/src/auth/auth.service.ts

```ts
function hashToken(token: string): string {
    return createHash('sha256').update(token).digest('hex');
}
  …
    private async createSession(userId: number, username: string)
    {
        const payload = { sub: userId, username };
        const access_token = await
            this.jwtService.signAsync(payload);
        // Sans jwtid, deux tokens crees dans la meme seconde
        // sont identiques
        const refresh_token = await
            this.jwtService.signAsync(payload, {
            secret: process.env.JWT_REFRESH_SECRET,
            expiresIn: '7d',
            jwtid: randomUUID(),
        });
  …
        await this.prisma.user.update({
            where: { id: userId },
            data: { hashedRefreshToken: hashToken(refresh_token)
                },
        });
```

Stocké haché

Seul le SHA-256 du refresh token est en base (User.hashedRefreshToken). Logout = on le met à null.

Pourquoi pas bcrypt ici ?

bcrypt ne lit que 72 octets : tous les JWT d'un même user commencent pareil. timingSafeEqual évite les attaques temporelles.

Côté front

fetchWithRefresh : sur un 401, appelle /auth/refresh une seule fois (promesse partagée), puis rejoue la requête.

AuthGuard : le JWT est lu dans un cookie httpOnly

backend/src/auth/guards/jwt-auth.guard.ts

```
canActivate(context: ExecutionContext,): boolean |
    Promise<boolean> {
    const req = context.switchToHttp().getRequest();
    const token = req.cookies['jwt'];
    if (!token)
        throw new UnauthorizedException();
    try {
        req.user = this.jwtService.verify(token);
    }
    catch {
        throw new UnauthorizedException("Guard problem");
    }
    return true;
}
```

backend/src/auth/cookies.ts

```
export const ACCESS_COOKIE_OPTIONS: CookieOptions = {
    httpOnly: true,
    secure: true,
    sameSite: 'strict',
};
```

`httpOnly`

Le JavaScript de la page ne peut pas lire le token : une faille XSS ne permet pas de le voler.

`secure`

Le cookie ne part qu'en HTTPS.

`sameSite: strict`

Pas envoyé depuis un autre site : protège du CSRF.

`WebSocket`

Même vérification au handshake : readCookie(headers.cookie, 'jwt') puis jwtService.verify.

Profil et avatar : on vérifie le contenu, pas le nom du fichier

backend/src/profile/avatar-storage.ts

```
// Le mimetype envoye par le navigateur peut mentir, on lit les premiers octets.
// Pas de SVG : il peut contenir du JavaScript.
function detectImageExtension(buffer: Buffer): string | null {
  if (buffer.subarray(0, 8).equals(Buffer.from([0x89, 0x50, 0x4e, 0x47, 0x0d,
      0x0a, 0x1a, 0x0a])))
    return 'png';
  if (buffer.subarray(0, 3).equals(Buffer.from([0xff, 0xd8, 0xff])))
    return 'jpg';
  if (buffer.subarray(0, 4).toString('latin1') === 'GIF8')
    return 'gif';
  if (buffer.subarray(0, 4).toString('latin1') === 'RIFF' && buffer.subarray(8,
      12).toString('latin1') === 'WEBP')
    return 'webp';
  return null;
}
```

backend/src/profile/profile.service.ts

```
const NAME_PATTERN = /^[a-zA-Z0-9_-]{3,20}$/;
export const BIO_MAX_LENGTH = 280;
  …
    if (name !== undefined) {
      if (typeof name !== 'string' || !NAME_PATTERN.test(name.trim()))
        throw new BadRequestException('Pseudo must be 3 to 20 letters,
            digits, _ or -');
      userData.name = name.trim();
```

Magic bytes : le type est lu dans les premiers octets ; le mimetype envoyé peut mentir.

Pas de SVG : il peut contenir du JavaScript.

Nom aléatoire (userId-UUID) et basename() à la suppression : pas de path traversal.

2 Mo max (limite de multer) ; l'ancien fichier est supprimé.

Pseudo unique : l'erreur Prisma P2002 devient un 409 lisible.

Avatar par défaut : les initiales du pseudo (AvatarFallback).

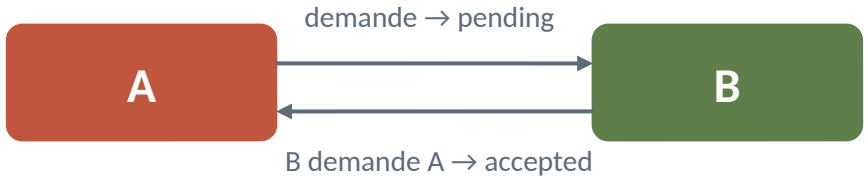
Amis : des demandes croisées sans course

backend/src/friends/friends.service.ts

```ts
private createOrAcceptRequest(userId: number, targetId: number):
    Promise<'pending' | 'accepted'> {
    return this.prisma.$transaction(async (tx) => {
        const existing = await tx.friendship.findFirst({ where: pairOf(userId,
            targetId) });
        if (!existing) {
            await tx.friendship.create({ data: { requesterId: userId,
                addresseeId: targetId } });
            return 'pending';
        }
        if (existing.status === 'accepted')
            throw new ConflictException('You are already friends');
        if (existing.requesterId === userId)
            throw new ConflictException('Request already sent');
        await tx.friendship.update({ where: { id: existing.id }, data: { status:
            'accepted' } });
        return 'accepted';
    }, { isolationLevel: Prisma.TransactionIsolationLevel.Serializable });
}
```

backend/prisma/schema.prisma

```
@@unique([requesterId, addresseeId])
```

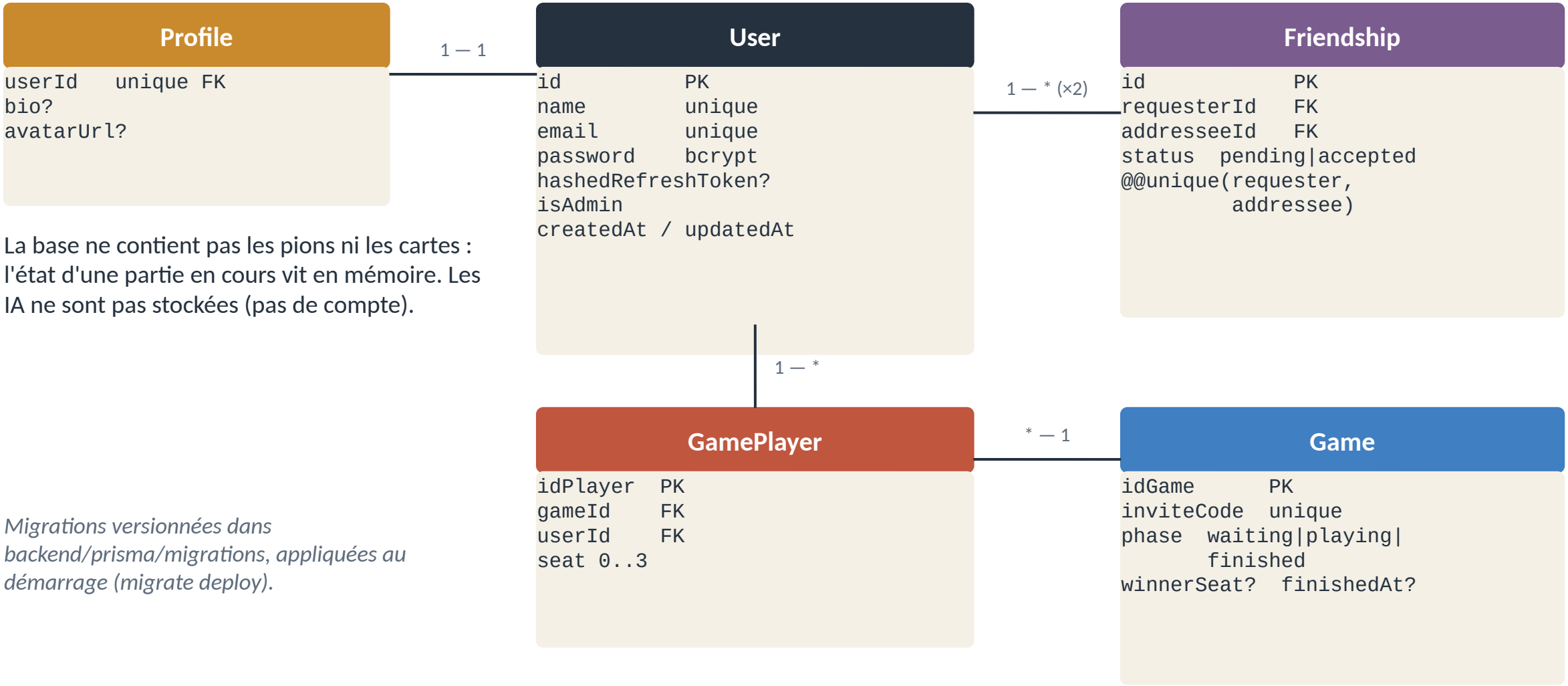


Course : A et B s'ajoutent au même instant. Postgres refuse une des deux transactions ; rejouée (3 essais max), elle accepte la demande de l'autre.

Présence : PresenceService compte les sockets par user. En ligne au 1er onglet, hors ligne au dernier.

Temps réel : chaque changement émet friends-changed dans la room user-<id> ; le front recharge la liste.

Base de données : 5 modèles Prisma, relations explicites



La base ne contient pas les pions ni les cartes : l'état d'une partie en cours vit en mémoire. Les IA ne sont pas stockées (pas de compte).

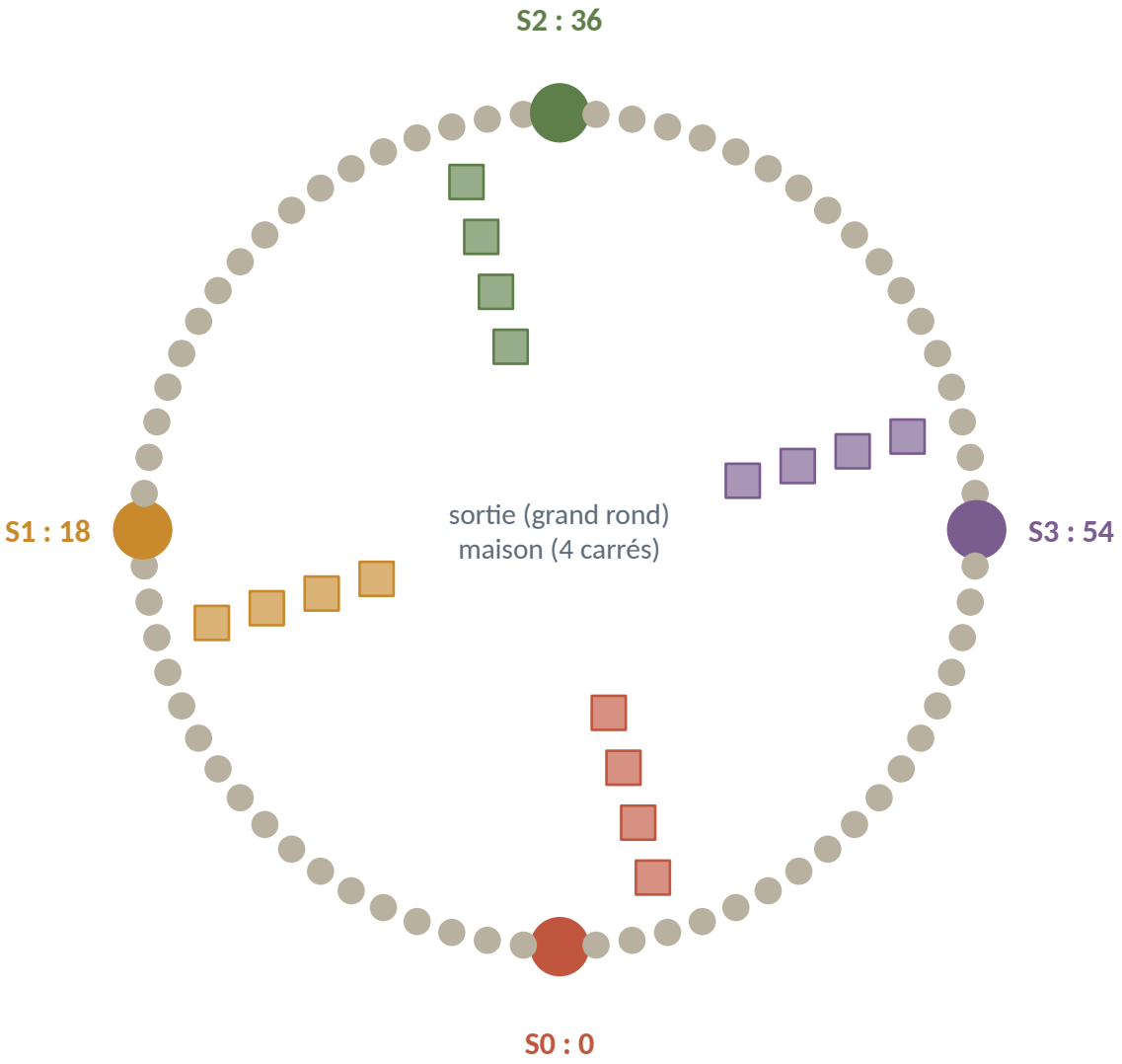
Migrations versionnées dans backend/prisma/migrations, appliquées au démarrage (migrate deploy).

3

Le moteur de jeu

backend/src/game/engine — des fonctions pures, sans Nest ni base

Le plateau : 72 cases en numérotation absolue



engine/board.ts

```ts
export function startCell(seat: Seat): number
{
  return (seat * CELLS_PER_SEAT);
}
  …
export function getStartHome(seat: Seat): number {
  let startingCell = startCell(seat);

  if (startingCell === 0)
    return BOARD_CELLS_COUNT - 3;
  return startingCell - 3;
}

export function homeCellIndex(seat: Seat, from: number, steps:
    number): number {
  const toEntry = mod(getStartHome(seat) - from, BOARD_CELLS_COUNT);

  return (steps - toEntry - 1);
}
```

- La même numérotation pour tout le monde : c'est le front qui tourne le plateau.
- mod() fait boucler 71 → 0, y compris en arrière (le 4).
- Pion : { zone: 'spawn' } | { zone: 'board', cell } | { zone: 'home', cell }.

Tout le jeu tient dans quelques types

backend/src/game/game.types.ts

```ts
export type CardId = `${CardColor}_${CardValue}`;
  …
export type PawnPosition =
  | { zone: 'spawn' }
  | { zone: 'board'; cell: number }
  | { zone: 'home'; cell: number };
  …
  | { type: 'enter'; cardId: CardId; pawnId: number }
  | { type: 'exit'; cardId: CardId; pawnId: number }
  | { type: 'swap'; cardId: CardId; pawnIdA: number; pawnIdB: number }
  // un seul morceau du 7 : un emit par morceau
  | { type: 'split'; cardId: CardId; pawnId: number; steps: number;
      enter: boolean }
  | { type: 'discard'; cardId: CardId };

/*
**********************************************************************
    **** */

  …
  action: GameAction;
  captured: Pawn[];
  newRound: boolean;
};

// Ce qu'un joueur a le droit de voir : pas la main des autres, pas la
// pioche.
```

Des ids, jamais des cases

Une action dit « quelle carte, quel pion ». Le serveur recalcule seul où il arrive.

Union discriminée

Le champ type permet à TypeScript de vérifier chaque cas du switch.

Une seule source

game.types.ts est copié dans le front (gameTypes.ts) : les deux côtés parlent le même langage.

Les cartes : ce que fait chaque valeur

Carte	Effet	Type d'action
As (1), Roi (13)	Sortir un pion du spawn, ou avancer	exit / move
4	Reculer de 4 cases	move (valeur −4)
5	Faire avancer un pion adverse	move
7	Répartir 7 pas entre ses pions, capture sur le passage	split (un emit par morceau)
Valet (11)	Échanger deux pions du plateau, n'avance jamais	swap
Autres	Avancer du nombre de cases	move / enter
Injouable	Défausser la carte	discard

realValueOf est utilisé par la validation ET par l'application du coup : elles ne peuvent pas diverger.

engine/deck.ts

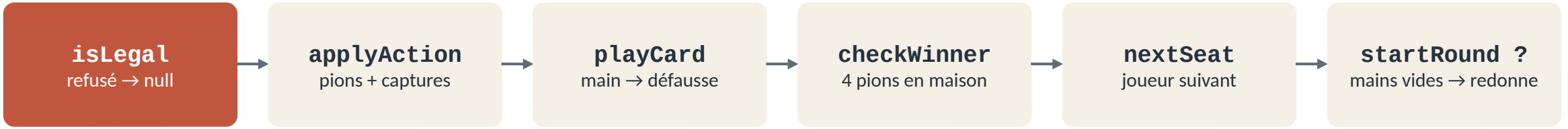
```
export function realValueOf(card: Card):
    number
{
  if (card.value === 4)
    return (-4);
  return (card.value);
}
```

engine/deck.ts

```
// Fisher-Yates : de la fin vers le debut, on
// echange chaque carte
// avec une carte tiree dans la partie pas
// encore melangee [0, i]
for (let i = shuffled.length - 1; i > 0; i--) {
  const j = Math.floor(Math.random() * (i +
      1)); // i + 1 pour que i soit atteignable
  [shuffled[i], shuffled[j]] = [shuffled[j],
      shuffled[i]];
}
```

Mélange Fisher-Yates : chaque ordre a la même probabilité.

playTurn : un coup, du contrôle à la redistribution



engine/state.ts

```
    return null;
  //lapplique
  …
  //la joue. Pendant un 7, la carte est deja posee depuis le premier morceau.
  …

  //ce qui vient de se passer, pour que le front puisse l'animer
  event = { seat, action, captured: result.captured, newRound: false };
  winner = checkWinner(game);
  if (winner !== null)
    return {game : {...game, phase: 'finished', winner}, event};
  //le 7 : tant qu'il reste des pas jouables, le meme joueur rejoue.
  …
  if (isRoundOver(game.players))
  {
    game = startRound(game);
    event.newRound = true;
  }

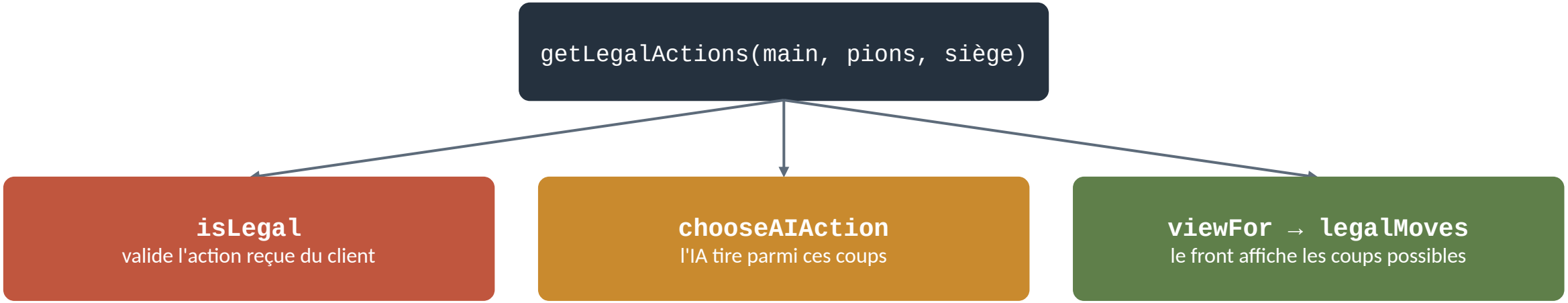
  return {game, event};
}
```

Retourne { game, event } ou null : jamais de mutation, l'ancien état reste intact.

event dit ce qui vient de se passer (qui, quelle action, quels pions capturés, nouvelle manche) : le front s'en sert pour animer.

Le 7 garde la main au même joueur tant qu'il reste des pas jouables (pendingSplit).

Une seule source de vérité : getLegalActions



engine/legalMoves.ts

```
export function isLegal(hand: Card[], pawns: Pawn[], seat: Seat,
    pendingSplit: Game['pendingSplit'], action: GameAction) : boolean {
  const legalMoves : GameAction[] = getLegalActions(hand, pawns, seat,
      pendingSplit);

  for (const legal of legalMoves)
  {
    if (sameAction(legal, action))
      return true;
  }
  return false;
```

On ne réimplémente jamais les règles pour valider : l'action reçue doit être identique (mêmes ids) à un des coups générés. Le client ne peut donc rien inventer.

Blocages et captures

engine/rules.ts

```ts
export function canMove(pawns: Pawn[], steps: number, pawn: Pawn) :
    boolean {

  if (pawn.position.zone !== 'board')
    return false;

  const allCells : number[] = pathCells(pawn.position.cell, steps);

  //si ya un spawn sur son chemin
  // si le pion a le meme seat que le spawn -> false
  for (const cell of allCells)
  {
    let spawnSeat: Seat | null = getSeatOfStartCell(cell);

    if (pawnAt(pawns, cell)?.playerSeat === spawnSeat)
      return false;
  }
  return true;
```

engine/apply.ts

```ts
export function captureAt(pawns: Pawn[], cell: number) :
    ActionResult {
  const victim = pawnAt(pawns, cell);

  if (victim === null)
    return {pawns, captured: []};
  return { pawns: setPawnPosition(pawns, victim.id,
      {zone: 'spawn'}), captured: [victim]};
}
```

Blocage

Un pion posé sur sa propre case de sortie bloque le passage à tous les autres pions.

Capture

Finir sur une case occupée renvoie l'occupant au spawn, même si c'est un de ses pions.

Le 7

Capture tout sur son chemin (captureAlongPath).

Le 7 : un coup découpé en morceaux

engine/split.ts

```ts
export function getSplitSteps(pawns: Pawn[], seat: Seat, cardId: CardId, left:
    number) : GameAction[] {
  …
  for (const pawn of pawnsOf(pawns, seat))
  {
    for (let steps = 1; steps <= left; steps++)
    {
      for (const enter of [false, true])
      {
        const after = simulateSplitStep(pawns, pawn, steps, enter);

        if (after !== null)
          splitSteps.push({type: 'split', cardId, pawnId: pawn.id, steps, enter});
      }
    }
  }
  return splitSteps;
```

engine/state.ts

```ts
  const canGoOn = left > 0 && getSplitSteps(game.pawns, seat, action.cardId,
      left).length > 0;

  if (canGoOn)
    return {game: {...game, pendingSplit: {cardId: action.cardId, left}}, event};
  game = {...game, pendingSplit: null};
}
```

1. Choisir un pion et un nombre de pas (1..restant)
2. Émettre { type: 'split', steps }
3. Le serveur simule le morceau (simulateSplitStep)
4. pendingSplit.left diminue, même joueur
5. Plus rien de jouable : pas restants perdus

L'IA : un coup légal, joué par une boucle côté serveur

engine/AI.ts

```ts
export function chooseAIAction(game: Game, seat: Seat) : GameAction | null {
  const legalMoves : GameAction[] = getLegalActions(game.players[seat].hand,
      game.pawns, seat, game.pendingSplit);
  …
  if (legalMoves.length === 0)
    return null;
  return legalMoves[Math.floor(Math.random() * legalMoves.length)];
```

backend/src/game/game.service.ts

```ts
while (this.botToPlay(gameId)) {
    const delay = BOT_MIN_DELAY_MS + Math.random() * (BOT_MAX_DELAY_MS -
        BOT_MIN_DELAY_MS);
    await new Promise(resolve => setTimeout(resolve, delay));
    const game = this.botToPlay(gameId);
    if (!game)
        break;
    const seat = game.currentSeat;
    const action = chooseAIAction(game, seat);
    const out = action && playTurn(game, seat, action);
    if (!out)
        break;
    this.commit(out.game, out.event);
}
```

Délai humain : 1 à 3 s aléatoires avant chaque coup, le même rythme pour tous les joueurs.

Un commit par coup : les joueurs voient chaque coup des bots.

Une seule boucle par partie (Set botLoops) : deux bots ne jouent jamais le même tour.

La partie est relue après chaque pause : un joueur a pu revenir ou partir.

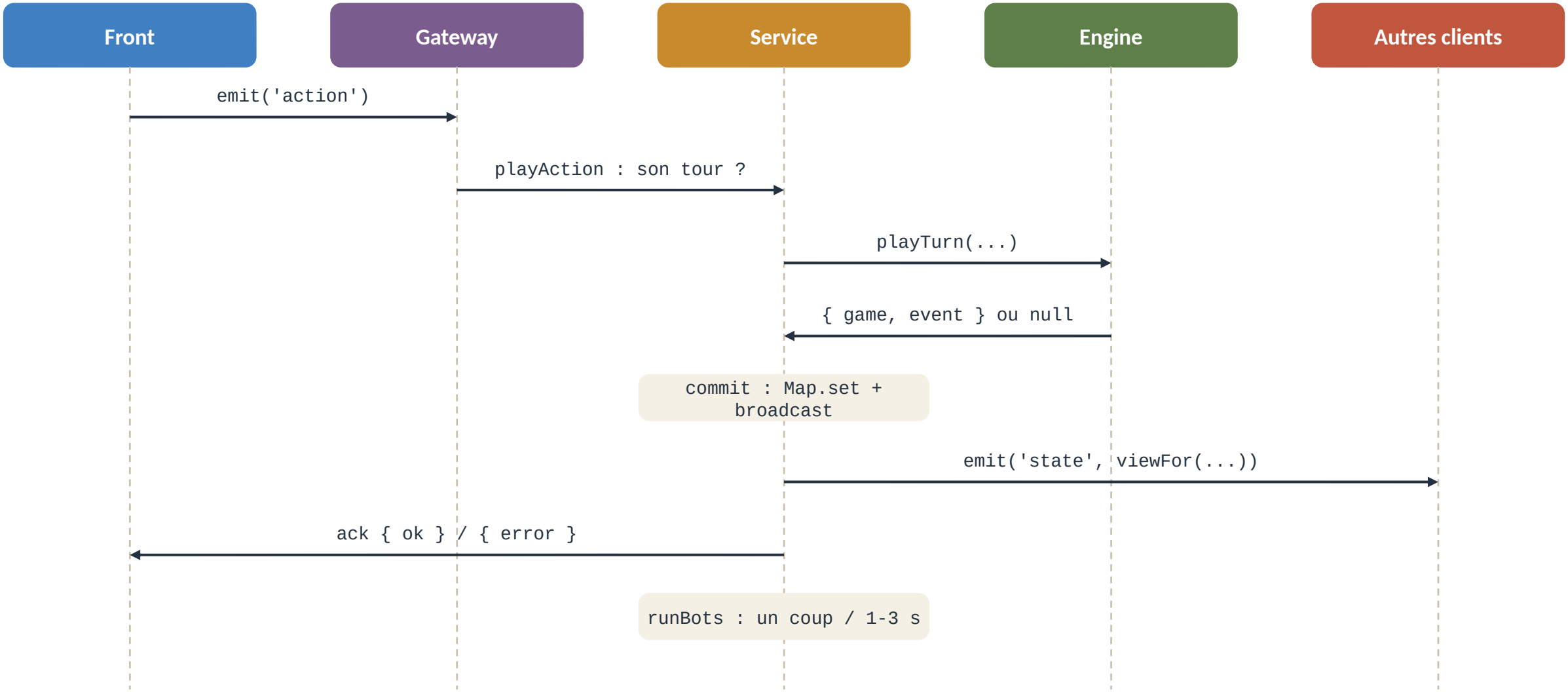
À savoir pour la soutenance : l'IA choisit au hasard parmi les coups légaux. Le module « AI Opponent » exige une IA qui peut gagner et qui imite un humain.

4

Temps réel

socket.io derrière nginx, sur /ws/socket.io

Le trajet d'un clic, du navigateur aux 4 joueurs



GameGateway : authentifie le socket, diffuse l'état

backend/src/game/game.gateway.ts

```
@WebSocketGateway({ path: '/ws/socket.io' })
  …
    afterInit() {
        this.gameService.setBroadcaster((game, event) => {
            for (const player of game.players) {
                if (isAI(player))
                    continue; // les bots n'ont pas de navigateur
                const message: StateMessage = { state: viewFor(game,
                    player.seat), event };
                this.server.to(`user-${player.userId}`).emit('state', message);
            }
        });
  …
    handleConnection(client: Socket) {
        const token = readCookie(client.handshake.headers.cookie, 'jwt');
        try {
            const payload = this.jwtService.verify(token ?? '');
            client.data.userId = payload.sub;
            // une room par user : tous ses onglets recoivent l'etat
            client.join(`user-${payload.sub}`);
        }
        catch {
            client.disconnect();
            return;
        }
        this.gameService.userConnected(client.data.userId);
    }
```

Une room par user

user-<id> : tous les onglets d'un joueur reçoivent l'état.

Service découplé

GameService ne connaît pas socket.io : la gateway lui injecte une fonction broadcast.

Jeton expiré ?

Socket coupé ; le front appelle /auth/refresh puis se reconnecte.

commit + viewFor : un état, quatre vues filtrées

game.service.ts

```ts
private commit(game : GameTypes.Game, event:
    GameTypes.GameEvent | null = null) {
    this.games.set(game.id, game);
    this.broadcast?.(game, event);
}
```

engine/view.ts

```ts
// La partie vue depuis un siege : on cache la pioche et la main
// des autres.
export function viewFor(game: Game, seat: Seat): GameView {
  const { drawPile, players, ...rest } = game;

  return {
    ...rest,
    mySeat: seat,
    myHand: players[seat].hand,
    players: players.map(({ hand, ...player }) => ({ ...player,
        handCount: hand.length })),
    drawPileCount: drawPile.length,
    legalMoves: legalMovesFor(game, seat),
  };
}
```



La pioche et les mains adverses ne quittent jamais le serveur : impossible de tricher en lisant le réseau.

Le protocole WebSocket complet

Sens	Message	Contenu	Quand
serveur → client	state	{ state: GameView, event: GameEvent | null }	À chaque commit, et en réponse à sync
serveur → client	no-game	—	sync alors que le joueur n'a pas de partie
client → serveur	sync	—	Arrivée sur une page de jeu, reconnexion
client → serveur	action	{ gameId, action } + ack { ok } | { error }	Le joueur joue
serveur → client	friends-changed	— (namespace /friends)	Demande, acceptation, suppression, connexion d'un ami

state est la vérité : le front affiche exactement ce qu'il reçoit.

event n'est qu'un indice pour animer ; il vaut null quand rien n'a été joué (sync).

Déconnexion : délai de grâce, puis l'IA prend le relais

Dernier onglet fermé
handleDisconnect

→

Délai de grâce
un refresh n'est pas un départ

→

isConnected = false
commit → les autres le voient

→

L'IA joue pour lui
isBotControlled(player)

→

Il revient
connexion → sync → il reprend

backend/src/game/game.service.ts

```
userDisconnected(userId: number) {
    clearTimeout(this.disconnectTimers.get(userId));
    this.disconnectTimers.set(userId, setTimeout(() => {
        this.disconnectTimers.delete(userId);
        this.setConnected(userId, false);
    }, RECONNECT_GRACE_MS));
}
  …
        // plus aucun humain pour regarder : personne ne reviendra, on arrete
        if (!updated.players.some(p => !GameTypes.isBotControlled(p))) {
            void this.abandonGame(updated.id);
            continue;
        }
        this.commit(updated);
        // si c'etait son tour, l'IA le joue pour lui
        if (!isConnected)
            void this.runBots(updated.id);
```

Plus aucun humain ?

abandonGame : phase finished, winnerSeat null. Différent d'une victoire.

Multi-onglets

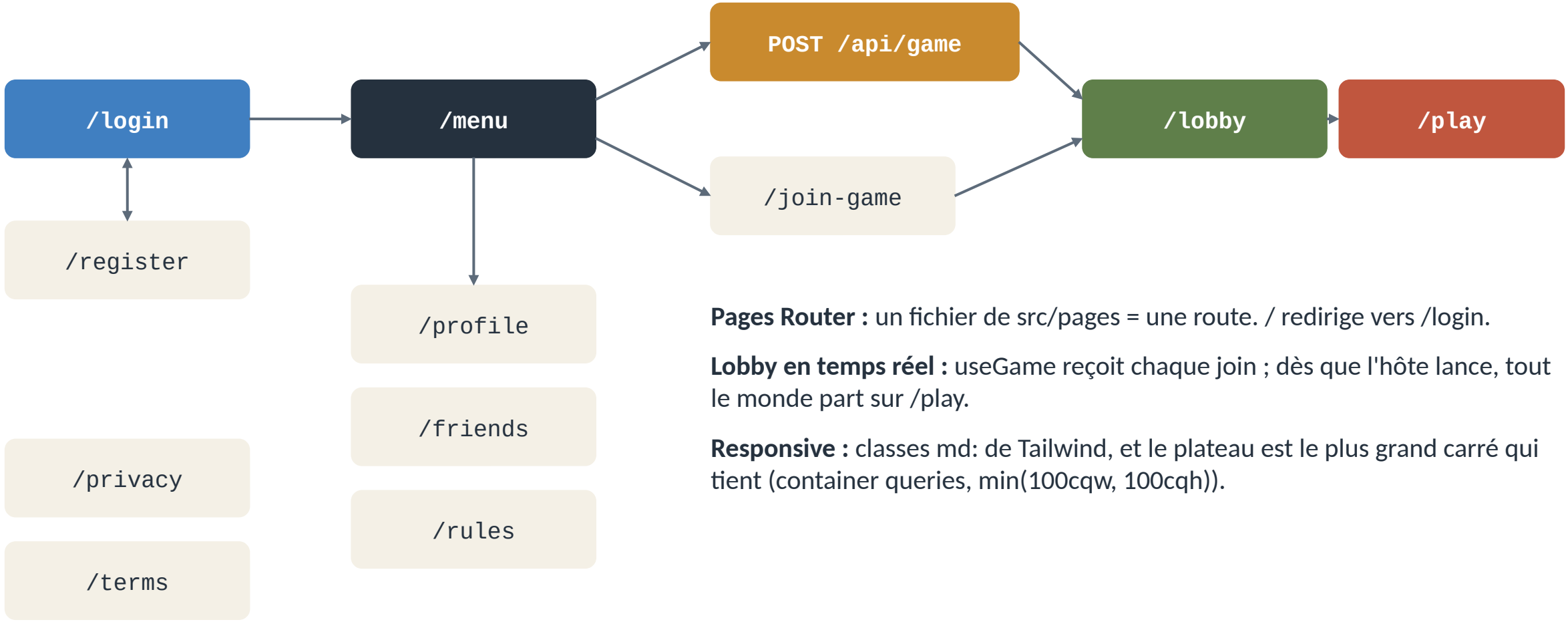
La gateway ne prévient le service que si plus aucun socket du user n'est dans sa room.

5

Frontend Next.js

Pages Router, hooks, plateau animé, design system

Les pages et le parcours d'un joueur



liens du footer, sur toutes les pages

Pages Router : un fichier de src/pages = une route. / redirige vers /login.

Lobby en temps réel : useGame reçoit chaque join ; dès que l'hôte lance, tout le monde part sur /play.

Responsive : classes md: de Tailwind, et le plateau est le plus grand carré qui tient (container queries, min(100cqw, 100cqh)).

useGame : un socket partagé, l'état poussé par le serveur

frontend/src/game/socket.ts

```ts
export function getSocket(): Socket {
  if (socket) {
    return socket;
  }

  socket = io({ path: "/ws/socket.io" });

  socket.on("disconnect", async (reason) => {
    if (reason !== "io server disconnect") {
      return;
    }
    const isRefreshed = await refreshAuth();
    if (isRefreshed) {
      socket?.connect();
    }
  });

  return socket;
```

frontend/src/game/useGame.ts

```ts
    setLastEvent(message.event);
    setHasNoGame(false);
    const event = message.event;
    …
}

socket.on("state", handleState);
socket.on("no-game", handleNoGame);
socket.on("connect", handleConnect);
if (socket.connected) {
  handleConnect();
}
```

Singleton

Créé au premier usage seulement : Next rend aussi les pages côté serveur, sans navigateur.

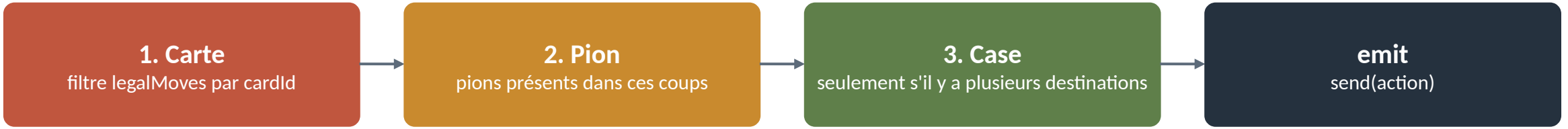
Resynchro

À chaque (re)connexion, le client émet sync et récupère l'état complet.

Pas de calcul

setGame(message.state) : le front affiche, il ne déduit rien.

useTurnChoice : carte → pion → case, sans connaître les règles



frontend/src/game/useTurnChoice.ts

```
function handleCardClick(clickedCardId: CardId) {
  if (clickedCardId === selectedCardId) {
    resetChoice();
    return;
  }
  …
  // a card that can do nothing else is discarded right away
  const moves = legalMoves.filter((move) => move.action.cardId === clickedCardId);
  if (moves.length === 1 && moves[0].action.type === "discard") {
    send(moves[0].action);
    return;
  }
  …
  // only one place to go: no need to ask where
  const pawnMoves = cardMoves.filter((move) => pawnIdsOf(move.action)[0] ===
      pawnId);
  if (!isJack && pawnMoves.length === 1) {
    send(pawnMoves[0].action);
    return;
  }
```

- Tout vient de game.legalMoves, calculé par le serveur pour ce joueur.
- Un seul coup possible ? Il part directement, sans clic de plus.
- Le Valet : premier pion, puis le pion avec lequel l'échanger.
- Une erreur du serveur (ack { error }) s'affiche sous les contrôles.

Chaque joueur voit le plateau depuis sa place

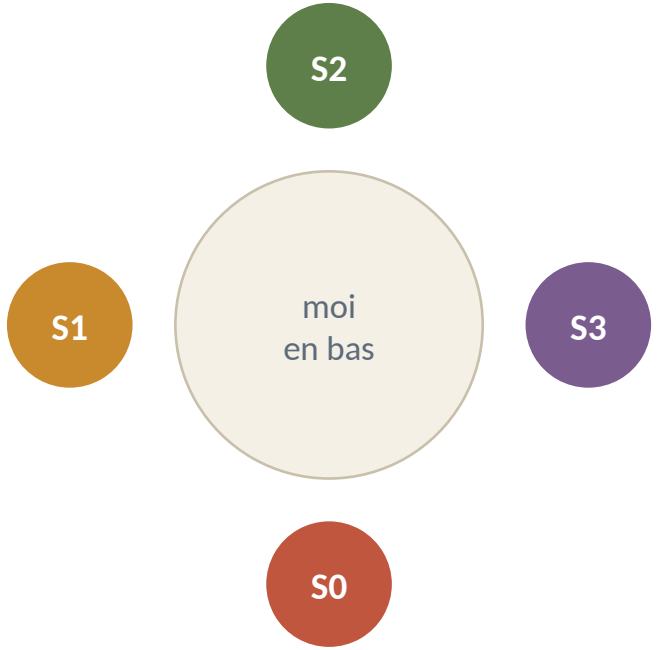
frontend/src/components/game/Board.tsx

```tsx
/** Quarter-turns (clockwise) that bring the arm of `seat` to the
    bottom. */
function rotationForSeat(seat: number): number {
  const quarters = (BOTTOM_ARM - armOfSeat(seat) + TEAMS_NUMBER) %
      TEAMS_NUMBER;
  return quarters * 90;
}
```

frontend/src/pages/play.tsx

```tsx
        previewMoves={turnChoice.cardMoves}
        onPawnClick={turnChoice.handlePawnClick}
      />
      <DestinationMarkers
        destinations={turnChoice.destinations}
        color={TEAM_COLORS[game.mySeat]}
        seat={game.mySeat}
        onClick={turnChoice.handleDestinationClick}
  …
        canPickCard={isMyTurn && !turnChoice.isSplitInProgress}
        onCardClick={turnChoice.handleCardClick}
      />
    )}
  </div>
</div>

{game.phase === "finished" && game.winner !== null && (
```



- Le serveur reste en coordonnées absolues ; le front tourne le plateau de quarts de tour.
- Les pions sont animés avec motion (event = qui a bougé, qui a été capturé).
- Un aperçu de trajet s'affiche au survol des pions jouables.

Style : Tailwind 4 + shadcn/ui, piloté par des tokens

frontend/src/styles/globals.css

```
@import 'tailwindcss';
@import "tw-animate-css";
@import "shadcn/tailwind.css";
  …
  --background: #E5DECF;
  --foreground: #25313E;
  …
  --primary: #407FC1;
  --primary-foreground: #FFFFFF;
  --secondary: #91B8E2;
  …
  /* Joueurs du plateau, dans l'ordre des equipes : haut,
      droite, bas, gauche. */
  --team-0: #C0563E;
  --team-1: #C98A2E;
  --team-2: #5F7F4A;
  --team-3: #7B5C8F;
```



background foreground primary secondary

Composants réutilisables

Button	Card	Field	Input
Label	Avatar	Breadcrumb	Separator
PageLayout	Header	Footer	Title
WornFrame	SeatBadge	CardRule	TiltCard

Bleu : shadcn/ui (components/ui) **Ocre :** composants maison

- Deux polices (Fraunces pour les titres, Public Sans pour le texte), exposées en variables CSS.
- Couleurs d'équipe en tokens --team-0..3, partagées par le plateau et le lobby.

6

Grille & modules

Où en est le projet face au sujet et à la feuille de correction

Partie obligatoire : état actuel du dépôt

Exigence	État	Preuve / commentaire
Frontend + backend + BDD	OK	Next.js, NestJS, PostgreSQL
Déploiement en une commande	OK	docker compose up --build, healthcheck postgres
HTTPS	À revoir	nginx TLS sur 8443, mais 3001 et 3002 sont publiés en clair dans le compose
Privacy Policy / Terms of Service	OK	/privacy et /terms, liens dans le footer, contenu réel
Framework CSS	OK	Tailwind 4 + shadcn/ui
.env ignoré + .env.example	Manquant	.env racine commité, pas de .gitignore racine, pas de .env.example
Schéma BDD et relations	OK	5 modèles Prisma, migrations versionnées
Mots de passe hachés et salés	OK	bcrypt, genSalt(10)
Validation front ET back	À revoir	OK pour profil, amis, mot de passe ; register et login non validés côté back
Multi-utilisateurs simultanés	OK	rooms socket.io, verrou « Not your turn », transaction sérialisable
README complet	Manquant	manquent : rôles, organisation, modules + points, contributions
Commits de tous les membres	OK	plusieurs auteurs (voir slide Git)

Modules : 13 points solides pour 14 requis

Module	Type	Pts	État	Ce qui le prouve dans le code
Framework front + back	Major	2	**Solide**	Next.js + NestJS
WebSockets temps réel	Major	2	**Solide**	commit → broadcast, rooms, reconnexion
ORM	Minor	1	**Solide**	Prisma 7 + migrations
Jeu web complet	Major	2	**Solide**	Tock : règles, victoire, parties live
Joueurs distants	Major	2	**Solide**	sync, délai de grâce, IA remplaçante
Multijoueur (3+)	Major	2	**Solide**	4 sièges synchronisés
Gestion utilisateurs standard	Major	2	**Solide**	profil, avatar + défaut, amis en ligne
Adversaire IA	Major	2	**Risqué**	IA aléatoire : pas « challenging »
Design system maison	Minor	1	**Risqué**	à documenter : palette, typo, icônes, 10+ comp.

13

points solides aujourd'hui

14

points requis

16

si l'IA et le design system passent

Un module non fonctionnel vaut 0 : viser plus que 14.

À corriger avant la soutenance

1 Conflit Git : à committer

Le conflit de game.service.ts est résolu en local (mode rapide retiré, IA à 1-3 s), mais le commit actuel contient encore les marqueurs : committer la correction.

2 .env dans le dépôt

Échec immédiat selon la grille. Ajouter un .gitignore racine, un .env.example, retirer .env du suivi et changer les mots de passe.

3 Register sans validation back

Le body entier est étalé dans prisma.user.create (…data) : un client peut envoyer isAdmin, qui vaut true par défaut.

4 Ports publiés en clair

3001 et 3002 sont exposés : tout doit passer par nginx en HTTPS.

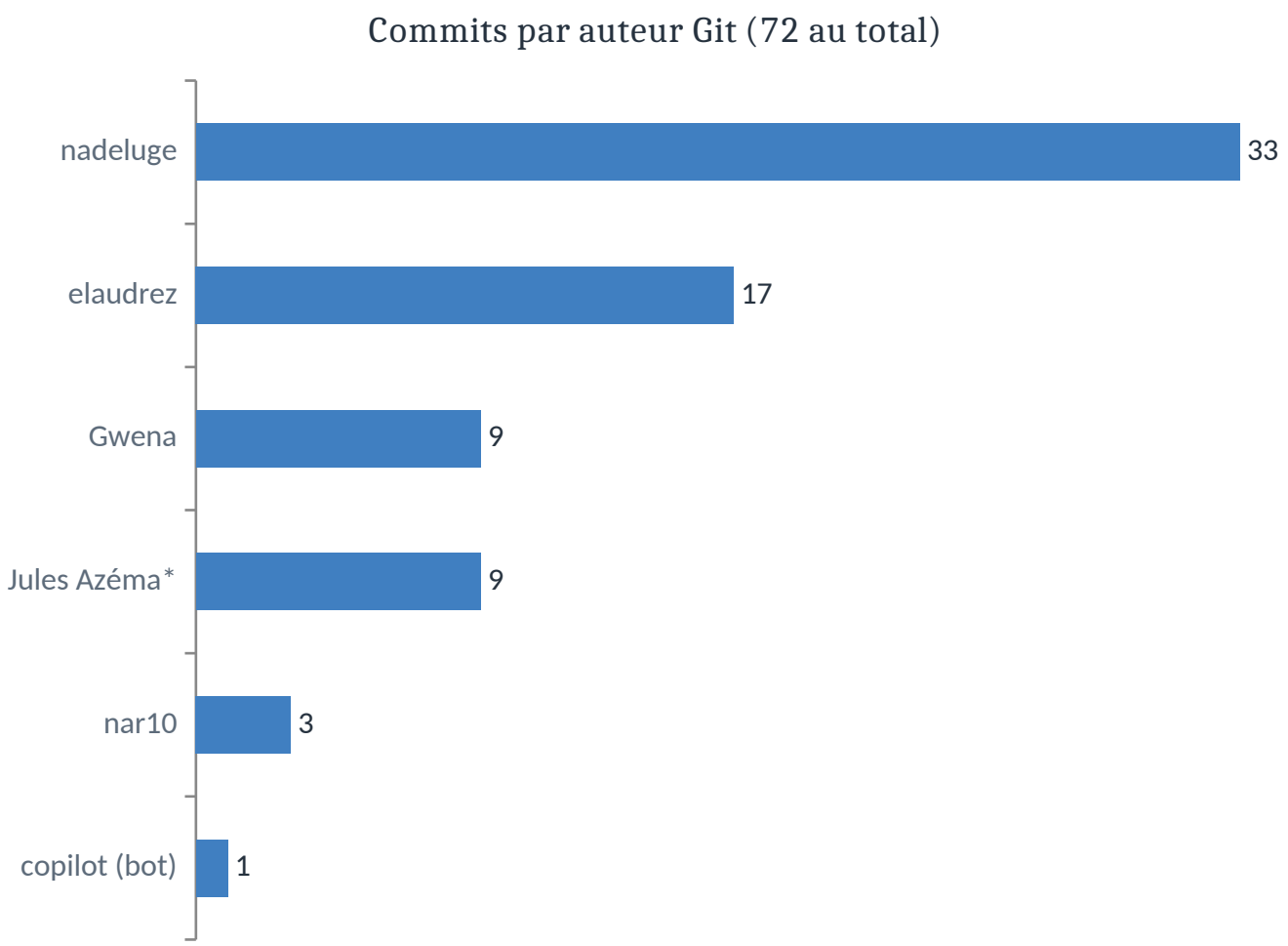
5 Fichiers parasites

.next/, frontend/frontend/.next, backend/cookies.txt, backend/POST, *.png~ sont commités.

6 README

Ajouter rôles (PO, PM, Tech Lead), organisation, modules + calcul des points, contributions. La section « reste à faire » est périmée.

Git : la contribution doit être visible pour chacun



* **Jules Azéma** regroupe 3 identités (jazema, julesazema, Jules Azéma) : unifier avec un .mailmap.

Préparer pour chaque membre : son rôle, une feature qu'il a codée, et un fichier à montrer.

Messages de commit : certains sont flous (« clean », « comment », « front changesx ») ; la grille demande des messages clairs.

Le commit du bot Copilot doit pouvoir être expliqué.

Questions probables du correcteur

? Comment les mots de passe sont-ils stockés ?

bcrypt avec un sel aléatoire par mot de passe ; le sel est dans le hash.

? Pourquoi des cookies et pas localStorage ?

httpOnly : un script injecté ne peut pas lire le token.

? Comment empêchez-vous la triche ?

Le serveur valide chaque action (isLegal) et cache les mains adverses (viewFor).

? Deux actions en même temps ?

Node traite un message à la fois ; le contrôle « Not your turn » refuse l'autre.

? Un joueur ferme l'onglet ?

Délai de grâce, puis l'IA joue pour lui ; à son retour, sync lui rend la main.

? Pourquoi l'état en mémoire ?

Rapide et simple. Prix à payer : un redémarrage du backend perd les parties.



À retenir

Le serveur décide, le moteur est pur, le front affiche ce qu'il reçoit.

Avant la soutenance : committer la résolution du conflit, sortir .env du dépôt, compléter le README.